

JavaScript Code Snippet Interview Questions

20 'What is the output?' questions testing basic syntax and fundamentals

Basics

Q1. What is the output?

```
console.log(typeof null);
```

Answer:

'object'. This is a long-standing bug in JavaScript kept for backward compatibility — null is a primitive, but typeof reports it as 'object'.

Q2. What is the output?

```
console.log(1 + '1');  
console.log('1' - 1);
```

Answer:

'11' then 0. The '+' operator triggers string concatenation when either operand is a string. The '-' operator only works numerically, so it coerces '1' to the number 1.

Q3. What is the output?

```
let x = 10;  
{  
  let x = 20;  
  console.log(x);  
}  
console.log(x);
```

Answer:

20 then 10. 'let' is block-scoped, so the inner x shadows the outer one only within its own block; the outer x is unaffected.

Q4. What is the output?

```
console.log(0.1 + 0.2 === 0.3);
```

Answer:

false. Floating-point numbers are stored in binary, and 0.1 + 0.2 actually evaluates to 0.30000000000000004, which is not strictly equal to 0.3.

Q5. What is the output?

```
console.log([] == false);  
console.log([] === false);
```

Answer:

true then false. With '==', [] is coerced to an empty string then to 0, and false becomes 0, so they're loosely equal. With '===', no coercion happens, and an array is never strictly equal to a boolean.

Hoisting & Scope

Q1. What is the output?

```
console.log(a);  
var a = 5;
```

Answer:

undefined. 'var' declarations are hoisted to the top of their scope and initialized as undefined, but the assignment (a = 5) stays in place, so it hasn't run yet at the time of the log.

Q2. What is the output?

```
console.log(b);  
let b = 5;
```

Answer:

ReferenceError: Cannot access 'b' before initialization. 'let' is hoisted but stays in the 'temporal dead zone' until its declaration line is reached, unlike 'var'.

Q3. What is the output?

```
for (var i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Answer:

0, 3, 3. Since 'var' is function-scoped, all three callbacks share the same 'i', which has already reached 3 by the time the callbacks run after the loop finishes.

Q4. What is the output?

```
for (let i = 0; i < 3; i++) {  
  setTimeout(() => console.log(i), 0);  
}
```

Answer:

0, 1, 2. 'let' creates a new binding of 'i' for each loop iteration, so each callback closes over its own separate value.

Q5. What is the output?

```
function foo() {  
  console.log(this);  
}  
foo();
```

Answer:

In non-strict mode, this logs the global object (e.g. 'window' in a browser); in strict mode ('use strict'), it logs undefined — because a plain function call has no explicit receiver for 'this'.

Functions & Closures

Q1. What is the output?

```
function outer() {  
  let count = 0;  
  return function () {  
    return ++count;  
  };  
}
```

```
const counter = outer();
console.log(counter());
console.log(counter());
```

Answer:

1 then 2. The returned inner function forms a closure over 'count', retaining and updating access to it across separate calls even though 'outer' has already finished executing.

Q2. What is the output?

```
const arr = [1, 2, 3];
const doubled = arr.map(function (n) {
  return n * 2;
});
console.log(doubled);
```

Answer:

[2, 4, 6]. 'map()' calls the given function once per element and returns a new array built from the return values, leaving the original array unchanged.

Q3. What is the output?

```
function add(a, b) {
  return a + b;
}
console.log(add(2));
```

Answer:

NaN. 'b' is not supplied, so it defaults to undefined, and 2 + undefined evaluates to NaN because undefined can't be meaningfully added as a number.

Q4. What is the output?

```
const greet = (name = 'Guest') => `Hello, ${name}`;
console.log(greet());
console.log(greet('Sam'));
```

Answer:

'Hello, Guest' then 'Hello, Sam'. The default parameter value is used only when the argument is omitted (or explicitly undefined).

Q5. What is the output?

```
function Person(name) {
  this.name = name;
}
const p = Person('Sam');
console.log(p);
console.log(name);
```

Answer:

undefined, then 'Sam' (in non-strict mode, in a browser). Without 'new', 'this' inside Person refers to the global object rather than a new instance, so the function returns nothing (undefined) and 'name' leaks onto the global object.

Objects, Arrays & Types

Q1. What is the output?

```
const obj = { a: 1, b: 2 };  
const { a, ...rest } = obj;  
console.log(a, rest);
```

Answer:

1 then { b: 2 }. Destructuring pulls out 'a' by name, and the rest operator ('...rest') collects all remaining own properties into a new object.

Q2. What is the output?

```
console.log([1, 2, 3] + [4, 5, 6]);
```

Answer:

'1,2,34,5,6'. The '+' operator on arrays first converts each array to a string via toString() (joining elements with commas), then concatenates the two resulting strings.

Q3. What is the output?

```
const a = { x: 1 };  
const b = a;  
b.x = 2;  
console.log(a.x);
```

Answer:

2. Objects are assigned and passed by reference, so 'b' points to the same underlying object as 'a' — mutating 'b' also affects what 'a' sees.

Q4. What is the output?

```
console.log([1, 2, 3].length === 3);  
console.log('abc'.length === 3);
```

Answer:

true then true. Both arrays and strings expose a 'length' property reflecting their number of elements or characters, respectively.

Q5. What is the output?

```
console.log(NaN === NaN);  
console.log(Object.is(NaN, NaN));
```

Answer:

false then true. NaN is defined to never equal itself under '===' (per the IEEE 754 spec), but Object.is() uses a stricter same-value comparison where NaN is considered equal to NaN.